TECHNISCHE UNIVERSITÄT
BERGAKADEMIE FREIBERG

Die Ressourcenuniversität. Seit 1765.

Computational Materials Science

HPC Project Assignment

Game of life on a hexahedral mesh using MPI

Submitted by:

Sarath Kumar Kancheti

(Marticulation No : 66024)

Professors in charge:

Prof. Dr. O. Rheinbach

1 Compiling instructions

1.1 Using mpicc manually

```
mpicc -fopenmp -O3 game_of_life.c -o game_of_life.out -lm
```

After compiling this, *game_of_life.out* an executable is created, which can be executed as follows:

```
mpirun -np 4 ./game_of_life.out <rows> <columns> <iterations>
```

2 Job submission using PBS

2.1 Job script (run_test_cases.pbs)

Below is the PBS script used to submit the MPI program to cluster and run the test cases:

```
#!/bin/bash
#PBS -N GoL_Project_Kancheti
#PBS -q teachingq
#PBS -l select=1:ncpus=60:mem=248gb:mpiprocs=60
#PBS -l walltime=00:30:00
#PBS -o scaling_results.txt
#PBS -e scaling_errors.txt

# 1. Setup Environment
module load gcc/11.4.0
module load openmpi/gcc/11.4.0/5.0.3

# Set working directory
PBS_O_WORKDIR=$HOME/HPC_Kancheti
cd $PBS_O_WORKDIR

# 2. Compile with Optimization
mpicc -O3 -march=native -fopenmp game_of_life.c -o game_of_life.out -lm
mpirun -np $RANKS ./game_of_life.out 10000 10000 100
```

A matrix of size 10000×10000 is used for 100 iterations for strong scaling test and a matrix of variable sizes (ROWS=(RANKS × 1000)) are used for weak scaling test.

2.2 Submitting the job

```
qsub run_test_cases.pbs
```

2.3 Monitoring the job

```
qstat uniqueJobID
```

2.4 Output

Two output text files are generated at the end of the simulation *scaling_errors.txt* and *scaling_results.txt*.

```
nano scaling_results.txt
```

A list of parameters will be displayed for the both test cases to check the performance.

```
[STRONG SCALING RESULTS]
Cores    Ranks    Threads    Time(s)      MCells/s
---------------------------------------------------

[WEAK SCALING RESULTS]
Ranks    Grid_Size          Time(s)
-----------------------------------------
```

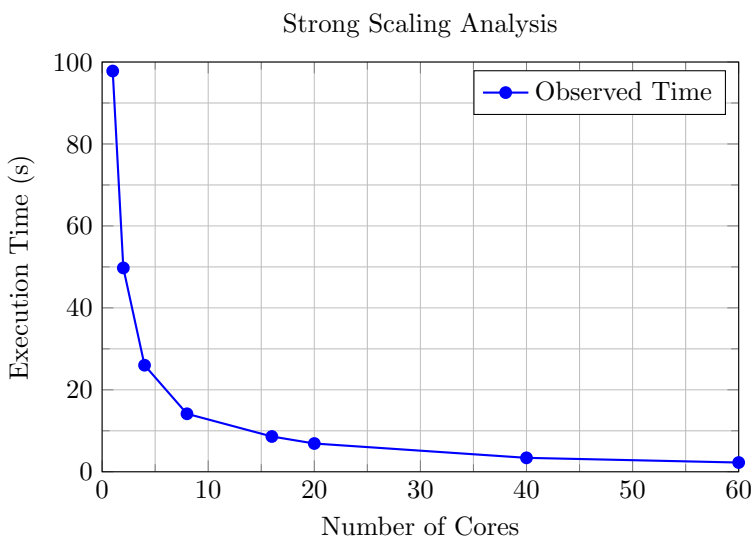
3 Results

3.1 Strong scaling

For strong scale testing a matrix of constant size 10000×10000 is used while varying the number of cores and the results can be observed below:

Table 1: Strong scaling results

Cores	Ranks	Threads	Time(s)	MCells/s
1	1	1	97.798	14.42
2	2	1	49.746	28.35
4	4	1	26.003	54.23
8	8	1	14.153	99.63
16	16	1	8.611	163.76
20	20	1	6.897	204.44
40	40	1	3.401	414.63
60	60	1	2.256	625.15



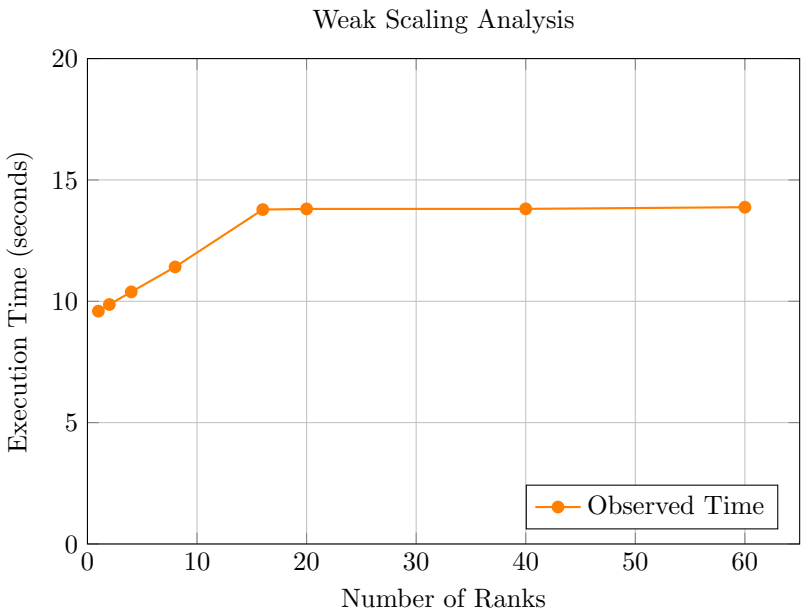
We can observe that execution time decreases with an increase in number of cores, indicating a good strong scaling performance.

3.2 Weak scaling

Weak scaling is analyzed by increasing both the matrix size and the number of processors proportionally as shown in the table.

Table 2: Weak Scaling Results: Grid Size vs. Number of Ranks

Ranks	**Grid Size**	**Execution Time (s)**
1	1000 × 10000	9.594
2	2000 × 10000	9.867
4	4000 × 10000	10.387
8	8000 × 10000	11.415
16	16000 × 10000	13.779
20	20000 × 10000	13.805
40	40000 × 10000	13.808
60	60000 × 10000	13.876



We can observe that execution time increases proportionally with increase in matrix size and number of ranks and becomes relatively stable. This indicates that the MPI implementation handles increased work load efficiently.

The best parallel scalability achieved is evidenced by the reduction of execution time from 97.798 seconds on 1 core to 2.256 seconds on 60 cores in the strong scaling test. Furthermore, in the weak scaling test, the implementation proved highly scalable by maintaining a stable execution time of approximately 13.8 seconds for a 60000 × 10000 grid on 60 ranks, compared to 9.594 seconds for a 1000 × 10000 grid on a single rank.

The simulation can be done for both dead border or circular board(torus) by changing the variable in main program.

```
config.boundary_type = TORUS;              /*  TORUS or DEAD_BORDER */
```